



第12章 继承与派生

张仁斌@合肥工业大学软件学院

本章主要内容



12.1 继承与派生



12.2 继承下的访问控制权

12.3 类的兼容性

12.4 派生类的构造函数与析构函数

12.5 派生类同名成员

12.6 虚基类——共享继承

12.7 实例：小型公司人员信息管理

12.8 作业与实践



继承与派生的基本概念

◆ 继承

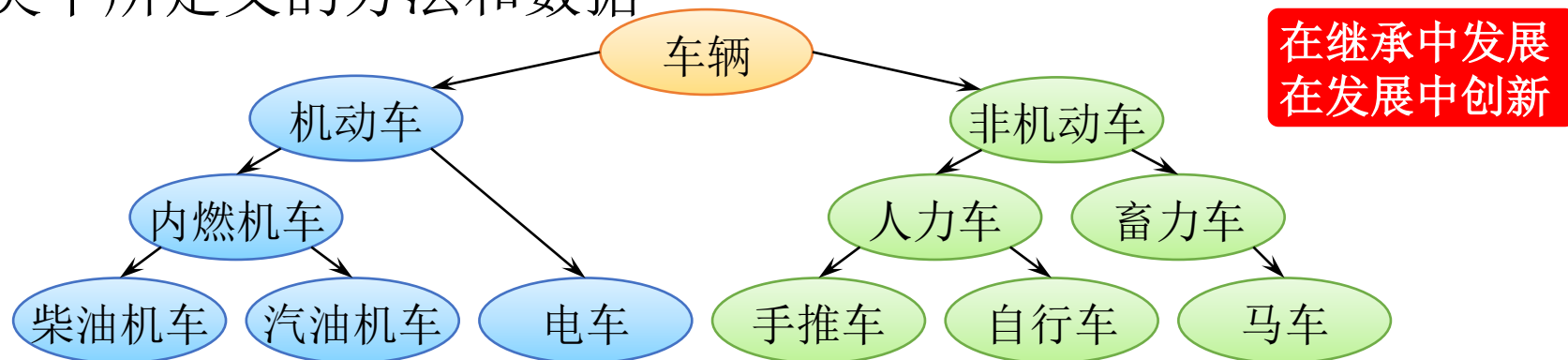
■ 是在已有类的基础上创建新的类，新类可以从一个或多个已有类中**继承成员函数或数据成员**，而且新类还可以**重新定义或添加新的数据和函数**

- 继承是使代码可以**复用**的最重要的手段（保存已有特性，实现代码重用），提高编程效率
- 不同于函数复用，继承是类设计层面的复用

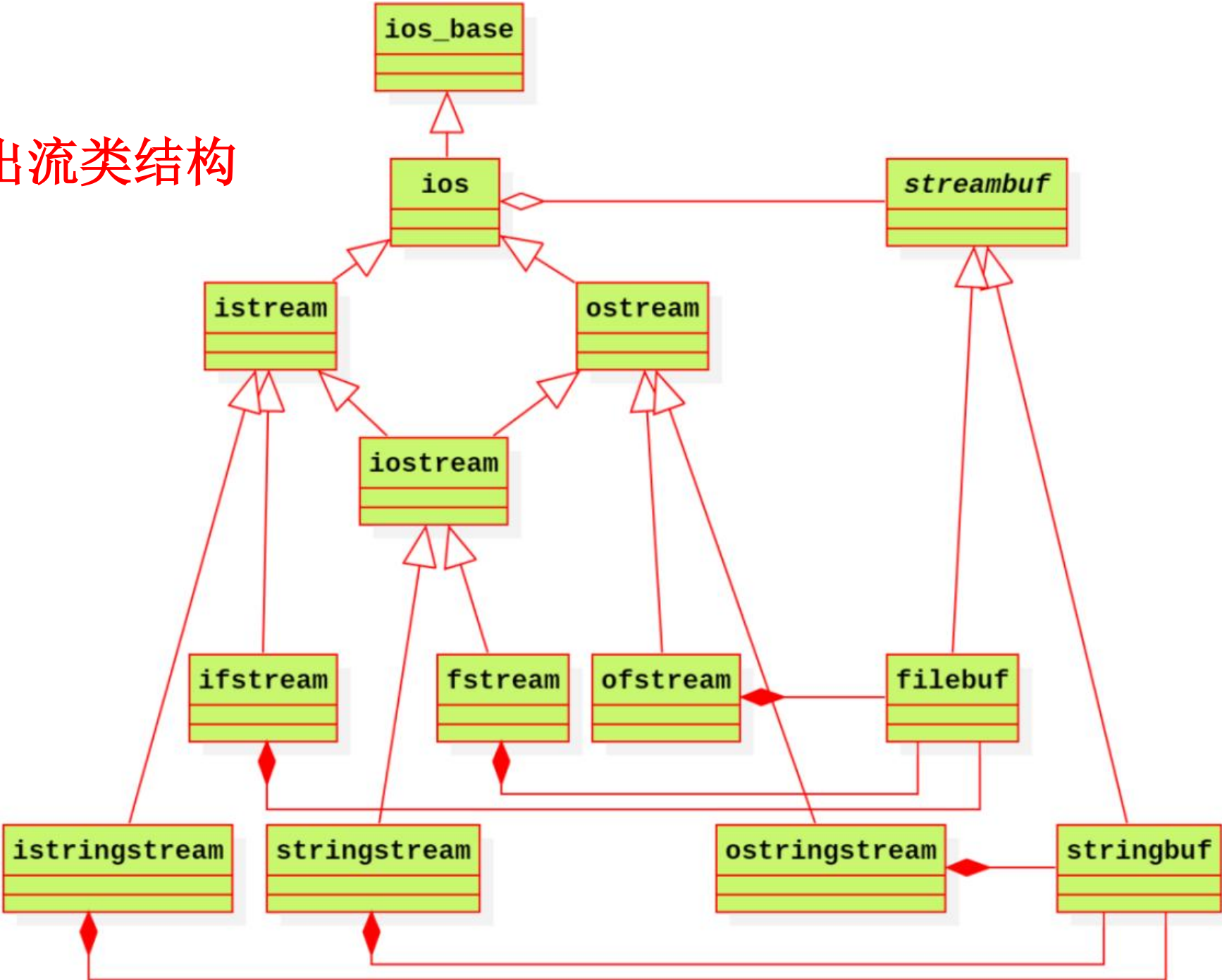
◆ 基类（父类）：用于派生新类的类

◆ 派生类（子类）：从任何已存在的类派生的新类

- 原有程序不能完全解决新问题时，需对原有程序进行改造，增加新特性
- 子类共享基类中所定义的方法和数据



C++输入输出流类结构



继承的种类与作用

◆ 单继承

- 一个子类只有一个**直接**父类

◆ 多继承

- 一个子类有两个或两个以上的**直接**父类

◆ 菱形继承（钻石继承）

- 多继承会导致存在菱形继承（不建议使用）

◆ 继承作用

- 吸收基类成员（**继承**）

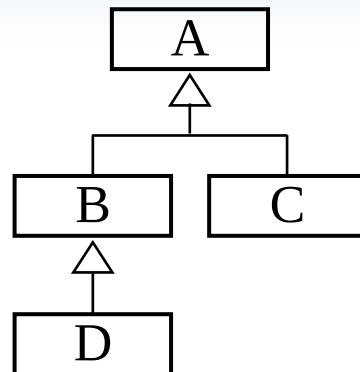
- 派生类包含**基类中除构造和析构造函数之外**的所有成员

- 添加新的成员（**创新**）

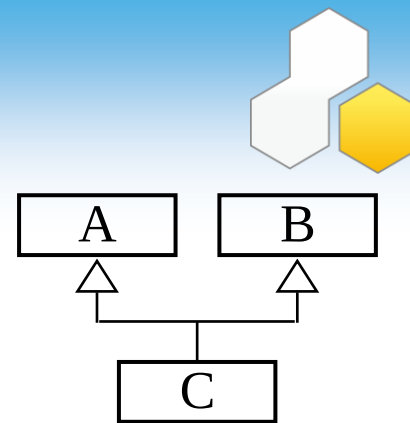
- 派生类添加新成员，是继承与派生的核心，保证派生类在功能上有所发展

- 改造基类成员（**发展**）

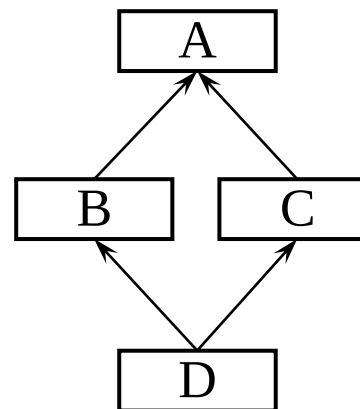
- 派生类声明和基类成员同名的新成员，新成员**覆盖**基类同名成员（若是成员函数，则参数表也要相同）



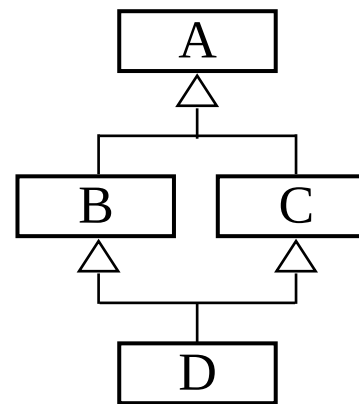
单继承



多继承



菱形继承





派生类的声明

◆ 单继承派生类的声明形式

三种继承方式（继承访问控制权）

（1）公有继承

（2）私有继承（默认）

（3）保护继承（一般不使用）

```
class 派生类名 : 派生/继承方式 基类名
{
    public:    公有成员说明列表;
    protected: 受保护成员说明列表;
    private:   私有成员说明列表;
};
```

■ 例如:

```
class Clock{
public:
    Clock(int h = 0, int m = 0, int s = 0);
    void setTime(int h, int m, int s);
    void showTime( );
private:
    int hour, minute, second;
};
```

```
class Date: public Clock {
public:
    Date(int y = 0, int m = 0, int d = 0);
    void setDate(int y, int m, int d);
    void showDate( );
private:
    int year, month, day; // 年/月/日
};
```

```
Date d;
d.setDate(2024, 12, 1);
d.showDate( );
d.setTime(20, 35, 50);
d.showTime( );
```

◆ 多继承派生类的声明形式

```
class 派生类名 : 继承方式 类名1, 继承方式 类名2, .....
{
    .....
};
```

本章主要内容



12.1 继承与派生

12.2 继承下的访问控制权



12.3 类的兼容性

12.4 派生类的构造函数与析构函数

12.5 派生类同名成员

12.6 虚基类——共享继承

12.7 实例：小型公司人员信息管理

12.8 作业与实践

基类中的访问权限在派生类中的变化



- ◆ C++中有三种基本的继承方式，**public**继承、**private**继承、**protected**继承，不同继承方式主要体现在**基类中的访问权限在派生类中的变化**
 - 派生类通过派生方式控制基类成员的访问权限
- ◆ **公有派生**：基类的公有成员和保护成员的访问属性在派生类中**不变**，而基类的私有成员，派生类不可访问
- ◆ **私有派生**：基类中的公有和保护成员**都以私有成员身份**出现在派生类中，基类的私有成员仍不可访问
- ◆ **保护派生**：在保护继承中，基类的公有和保护成员**都以保护成员的身份**出现在派生类中，而基类的私有成员不可访问

类成员 \ 继承方式	public继承	protected继承	private继承
基类的public成员	派生类的public成员	派生类的protected成员	派生类的private成员
基类的protected成员	派生类的protected成员	派生类的protected成员	派生类的private成员
基类的private成员	在派生类中不可见	在派生类中不可见	在派生类中不可见



◆ 派生类对基类成员的访问有以下两种

- 内部访问（派生类成员）：由派生类中新增成员对基类继承来的成员的访问
- 外部访问（派生类对象）：在派生类外部，通过派生类的对象对从基类继承来的成员的访问

◆ 结论

- 派生类成员：可访问基类中的公有成员和保护成员
- 派生类对象：可访问公有继承下基类中的公有成员

公有继承



继承方式	基类中访问控制权	派生类中访问控制权
public	public	public
	protected	protected
	private	inaccessible

```
class Base
{
public:
    int a1;
protected:
    int a2;
private:
    int a3;
};
```

```
class PublicClass: public Base
{
public:
    void test( )
    {
        a1 = 1; // a1仍然保持public
        a2 = 2; // a2仍然保持protected
        a3 = 3; × 基类私有成员不可访问
    }
};
```

派生类**成员**能访问基类中的public/protected成员，但不能访问private成员

```
int main( )
{
    PublicClass pc;
    pc.test();
    cout << pc.a1;
    cout << pc.a2; ×
    cout << pc.a3; ×
}
```

派生类**对象**不可访问基类中protected和private成员（**对象**只能访问public）

私有继承



继承方式	基类中访问控制权	派生类中访问控制权
private	public	private
	protected	private
	private	inaccessible

```
class Base
{
public:
    int a1;
protected:
    int a2;
private:
    int a3;
};
```

```
class PrivateClass: private Base
{
public:
    void test( )
    {
        a1 = 1; // a1被转变为private
        a2 = 2; // a2被转变为private
        a3 = 3; × 基类私有成员不可访问
    }
};
```

派生类**成员**能访问基类中的public/protected成员，但不能访问private成员

```
int main( )
{
    PrivateClass pc;
    pc.test();
    cout << pc.a1; ×
    cout << pc.a2; ×
    cout << pc.a3; ×
}
```

派生类**对象**不可访问基类中任何成员（**对象**只能访问public成员）

保护继承

继承方式	基类中访问控制权	派生类中访问控制权
protected	public	protected
	protected	protected
	private	inaccessible

- (1)派生类成员而言，除了基类中私有成员，其他成员都可访问；
- (2)派生类对象而言，只有公有继承下基类中的公有成员可访问；
- (3)不论何种继承方式，基类中的私有成员是不可访问的

```
class Base
{
public:
    int a1;
protected:
    int a2;
private:
    int a3;
};
```

```
class ProtectedClass: protected Base
{
public:
    void test( )
    {
        a1 = 1; // a1被转变为protected
        a2 = 2; // a2仍然保持protected
        a3 = 3; ✗基类私有成员不可访问
    }
};
```

派生类**成员**能访问基类中的public/protected成员，但不能访问private成员

```
int main( )
{
    ProtectedClass pc;
    pc.test();
    cout << pc.a1; ✗
    cout << pc.a2; ✗
    cout << pc.a3; ✗
}
```

派生类**对象**不可访问基类中任何成员（**对象**只能访问public成员）

示例：公有继承



Point

```
- x, y;  
+ void setPoint(float x , float y )  
+ void move(float xOff, float yOff)  
+ float getX ()  
+ float getY()
```

```
class Point {  
public:  
    void setPoint(float xx=0, float yy=0) { x = xx; y = yy; }  
    void move(float xOff, float yOff) { x += xOff; y += yOff; }  
    float getX() const { return x; }  
    float getY() const { return y; }  
private:  
    float x, y;  
};
```

Rectangle

继承

增加

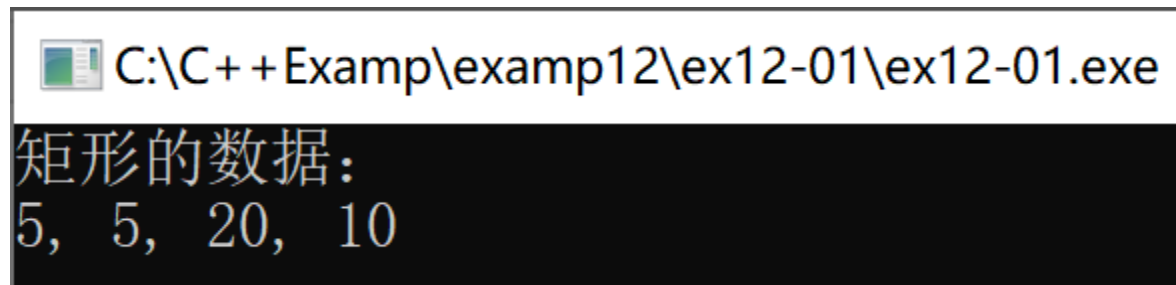
```
× x, y  
+ void setPoint(float x = 0, float y = 0)  
+ void move(float xOff, float yOff)  
+ float getX()  
+ float getY()  
+ void setRectangle(float, float, float, float )  
+ float getH() const  
+ float getW() cons  
- float w, h;
```

```
class Rectangle: public Point {  
public:  
    void setRectangle(float x, float y, float w, float h) {  
        setPoint(x, y); // 调用基类公有成员函数  
        this->w = w;    this->h = h;  
    }  
    float getH() const { return h; }  
    float getW() const { return w; }  
private:  
    float w, h;  
};
```



```
#include <iostream>
#include "Rectangle.h"
using namespace std;

int main()
{
    Rectangle rect;
    rect.setRectangle(2, 3, 20, 10);
    rect.move(3, 2);
    cout << "矩形的数据: \n";
    cout << rect.getX() << ", "
         << rect.getY() << ", "
         << rect.getW() << ", "
         << rect.getH() << endl;
    return 0;
}
```



C:\C++Examp\examp12\ex12-01\ex12-01.exe

矩形的数据:
5, 5, 20, 10

示例：私有继承



Point

```
- x, y;  
+ void setPoint(float x , float y )  
+ void move(float xOff, float yOff)  
+ float getX ()  
+ float getY()
```

```
class Point {  
public:  
    void setPoint(float xx=0, float yy=0) { x = xx; y = yy; }  
    void move(float xOff, float yOff) { x += xOff; y += yOff; }  
    float getX() const { return x; }  
    float getY() const { return y; }  
private:  
    float x, y;  
};
```

Rectangle

继承

增加

```
× x, y  
- void setPoint(float x = 0, float y = 0)  
- void move(float xOff, float yOff)  
- float getX()  
- float getY()  
+ void setRectangle(float, float, float, float )  
+ float getH() const  
+ float getW() const  
- float w, h;
```

```
class Rectangle: private Point {  
public:  
    void setRectangle(float x, float y, float w, float h) {  
        setPoint(x, y); // 调用基类公有成员函数  
        this->w = w;    this->h = h;  
    }  
    float getH() const { return h; }  
    float getW() const { return w; }  
private:  
    float w, h;  
};
```



```
#include <iostream>
#include "Rectangle.h"
using namespace std;

int main()
{
    Rectangle rect;
    rect.setRectangle(2, 3, 20, 10);
    rect.move(3, 2); ✗
    cout << "矩形的数据: \n";
    cout << rect.getX() << ", " ✗
        << rect.getY() << ", " ✗
        << rect.getW() << ", "
        << rect.getH() << endl;
    return 0;
}
```

main.cpp Point.h Rectangle.h

```
1  #ifndef POINT_H
2  #define POINT_H
3
4  class Point
5  {
6  public:
7      void setPoint(float xx=0, float yy=0) { x = xx; y = yy; }
8      void move(float xOff, float yOff) { x += xOff; y += yOff; }
9      float getX() const { return x; }
10     float getY() const { return y; }
11 private:
12     float x, y;
13 };
14
15 #endif
```

Resources Compile Log Debug Find Results Close

	Message
examp12\ex12-01\Point.h	[Error] 'void Point::move(float, float)' is inaccessible
examp12\ex12-01\main.cpp	[Error] within this context
examp12\ex12-01\main.cpp	[Error] 'Point' is not an accessible base of 'Rectangle'
examp12\ex12-01\Rectangle.h	In file included from Rectangle.h
examp12\ex12-01\main.cpp	from main.cpp
examp12\ex12-01\Point.h	[Error] 'float Point::getX() const' is inaccessible
examp12\ex12-01\main.cpp	[Error] within this context



◆ 派生类的**对象**不能访问已转变为私有成员函数的基类公有成员函数，在派生类中增加公有成员函数访问基类的该函数，以便派生类的对象间接访问基类的该函数

```
class Rectangle: private Point {
public:
    void setRectangle(float x, float y, float w, float h) {
        setPoint(x, y); // 调用基类公有成员函数
        this->w = w;
        this->h = h;
    }
    float getPX() const { return getX(); }
    float getPY() const { return getY(); }
    float getH() const { return h; }
    float getW() const { return w; }
private:
    float w, h;
};
```

多继承

```
class A {
public:
    void setA(int x) { a = x; }
    void showA() const { cout << a << endl; };
private:
    int a;
};
class B {
public:
    void setB(int x) { b = x; }
    void showB() const { cout << b << endl; };
private:
    int b;
};
class C : public A, private B {
public:
    void setC(int x, int y, int z) { setA(x), setB(y), c = z; }
    void showC() const { cout << c << endl; }
private:
    int c;
};
```

不能通过对象访问派生类
继承得到的私有成员函数

D:\Edu-CPP\chap12\ex12-02.exe

5
8
9

```
int main() {
    C obj;
    obj.setA(5);
    obj.showA();
    obj.setC(6, 7, 8);
    obj.showC();
    obj.setB(9);
    obj.showB();
    return 0;
}
```

Compile Log



Debug



Find Results

Message

In function 'int main()':

[Error] 'void B::setB(int)' is inaccessible

[Error] within this context

[Error] 'B' is not an accessible base of 'C'

[Error] 'void B::showB() const' is inaccessible

[Error] within this context

[Error] 'B' is not an accessible base of 'C'

本章主要内容



12.1 继承与派生

12.2 继承下的访问控制权

12.3 类的兼容性



12.4 派生类的构造函数与析构函数

12.5 派生类同名成员

12.6 虚基类——共享继承

12.7 实例：小型公司人员信息管理

12.8 作业与实践



- ◆通过公有继承，派生类拥有基类中除构造函数、析构函数之外的所有成员，且所有成员的访问控制属性也和基类基本相同（私有除外），因此，**公有派生类具备基类的所有功能**
- ◆类的**兼容性**：凡需要基类对象的地方，都可用公有派生类对象来替代

```
class Base1 {  
public:  
    void display() const { cout << "Base1::display()\n"; }  
};  
  
class Base2: public Base1 {  
public:  
    void display() const { cout << "Base2::display()\n"; }  
};  
  
class Derived: public Base2 {  
public:  
    void display() const { cout << "Derived::display()\n"; }  
};
```

```
void fun(Base1 *ptr) {  
    ptr->display();  
}  
  
int main() {  
    Base1 base1;  
    Base2 base2;  
    Derived derived;  
    fun(&base1);  
    fun(&base2);  
    fun(&derived);  
    return 0;  
}
```

允许将后代类的对象赋值给祖先类，但仅发挥祖先类的作用

D:\Edu-CPP\chap12\ex12-03.exe

```
Base1::display()  
Base1::display()  
Base1::display()
```



◆ 赋值兼容规则是指在需要基类对象的任何地方都可以使用公有派生类的对象来替代

- 公有派生类的对象可以被当作基类对象使用，反之则禁止
- 通过基类对象名、指针只能使用从基类继承的成员
- 此规则只适用于公有派生，只有公有派生类才能兼容基类类型
- 派生类的对象可以隐含转换为基类对象

➤ obj1 = obj2;

- 派生类的对象可以初始化基类的引用

➤ Base1 &obj = obj2;

- 派生类的指针可以隐含转换为基类的指针

➤ p = p1;

Base1	obj1, *p;
Base2	obj2;
Derived	obj3, *p1;

p = &obj1; ✓
p = &obj2; ✓
p = &obj3; ✓
obj1 = obj2; ✓
obj2 = obj3; ✓

p1 = &obj1; ✗
p1 = &obj2; ✗
p1 = &obj3; ✓
p = p1; ✓

本章主要内容



12.1 继承与派生

12.2 继承下的访问控制权

12.3 类的兼容性

12.4 派生类的构造函数与析构函数



12.5 派生类同名成员

12.6 虚基类——共享继承

12.7 实例：小型公司人员信息管理

12.8 作业与实践

构造函数、析构函数的调用次序



- ◆ 基类和派生类都有各自的构造函数和析构函数。在派生类创建对象和撤销对象时，基类和派生类的构造函数和析构函数调用规则是怎样的？
- ◆ 构造函数的调用次序
 - 先调用其**基类**的构造函数
 - 再调用本类中**对象成员**的构造函数（若无对象成员，则调过本步骤）
 - 最后才调用**本类**的构造函数
- ◆ 析构函数的调用次序（与构造函数调用次序**反序**）
 - 先调用本类的析构函数
 - 再调用本类中对象成员的析构函数
 - 最后才调用其基类的析构函数



```
class Base1 {
public:
    Base1() { cout << "Base1() executes...\n"; }
    ~Base1() { cout << "~Base1() executes...\n"; }
};
```

```
class Derived1:public Base1 {
public:
    Derived1() { cout << "Derived1() executes...\n"; }
    ~Derived1() { cout << "~Derived1() executes...\n"; }
};
```

```
int main()
{
    Derived1 d;
    return 0;
}
```

创建派生类对象时，先执行基类构造函数，再执行派生类的构造函数；析构函数调用顺序反之

D:\Edu-CPP\chap12\ex12-04.exe

```
Base1() executes...
Derived1() executes...
~Derived1() executes...
~Base1() executes...
```

```
class Base2 {
public:
    Base2() { cout << "Base2() executes...\n"; }
    ~Base2() { cout << "~Base2() executes...\n"; }
};
```

```
class Derived2:public Base2, public Base1 {
public:
    Derived2() { cout << "Derived2() executes...\n"; }
    ~Derived2() { cout << "~Derived2() executes...\n"; }
};
```

```
int main()
{
    Derived2 d;
    return 0;
}
```

若有多个基类，基类构造函数调用顺序按照它们被声明的顺序

D:\Edu-CPP\chap12\ex12-04B.exe

```
Base2() executes...
Base1() executes...
Derived2() executes...
~Derived2() executes...
~Base1() executes...
~Base2() executes...
```




```
class Base1 {
public:
    Base1() { cout << "Base1() executes...\n"; }
    ~Base1() { cout << "~Base1() executes...\n"; }
};

class Base2 {
public:
    Base2() { cout << "Base2() executes...\n"; }
    ~Base2() { cout << "~Base2() executes...\n"; }
};

class Derived:public Base1, public Base2 {
public:
    Derived() { cout << "Derived() executes...\n"; }
    ~Derived() { cout << "~Derived() executes...\n"; }
    Base1 b1;
    Base2 b2;
};
```

```
int main()
{
    Derived d;
    return 0;
}
```

先调基类构造函数，再对象成员构造函数，最后派生类构造函数；析构函数调用顺序反之

D:\Edu-CPP\chap12\ex12-04C.exe

```
Base1() executes...
Base2() executes...
Base1() executes...
Base2() executes...
Derived() executes...
~Derived() executes...
~Base2() executes...
~Base1() executes...
~Base2() executes...
~Base1() executes...
```

向基类构造函数传参



◆ 引例

```
class Base {  
    int x;  
public:  
    Base(int a) { x = a; cout << "Base(int) executes...\n"; }  
    ~Base() { cout << "~Base() executes...\n"; }  
    void getX(){cout << "Base::x = " << x << endl; }  
};
```

```
class Derived:public Base {  
public:  
    Derived() { cout << "Derived() executes...\n"; }  
    ~Derived() { cout << "~Derived() executes...\n"; }  
};
```

```
int main() {  
    Derived d;  
    return 0;  
}
```

基类只有带参构造函数，如何向基类构造函数传递参数？

◆ 解决方案

- 方案1：基类Base增加一个无参的或带默认值的构造函数
- 方案2：在Derived的构造函数初始化列表中显式调用Base的某个构造函数

```
class Derived:public Base {  
public:  
    Derived() { cout << "Derived() executes...\n"; }  
    ~Derived() { cout << "~Derived() executes...\n"; }  
};  
  
int main()  
{  
    Derived d;  
    return 0;  
}
```

Compile Log Debug Find Results Close

Message

In constructor 'Derived::Derived()':

[Error] no matching function for call to 'Base::Base()'

◆ 方案1

```
class Base {
int x;
public:
    Base(int a) { x = a; cout << "Base(int) executes...\n"; }
    Base() {x = 0; cout << "Base() executes...\n"; }
    ~Base() { cout << "~Base() executes...\n"; }
    void getX(){cout << "Base::x = " << x << endl; }
};

class Derived:public Base {
public:
    Derived( ){getX(); cout << "Derived() executes...\n"; }
    ~Derived() { cout << "~Derived() executes...\n"; }
};

int main() {
    Derived d;
    return 0;
}
```

D:\Edu-CPP\chap12\ex12-05B.exe

```
Base() executes...
Base::x = 0
Derived() executes...
~Derived() executes...
~Base() executes...
```

◆ 方案2

```
class Base {
int x;
public:
    Base(int a) { x = a; cout << "Base(int) executes...\n"; }
    ~Base() { cout << "~Base() executes...\n"; }
    void getX(){cout << "Base::x = " << x << endl; }
};

class Derived:public Base {
public:
    Derived(int i):Base(i) {
        getX(); cout << "Derived(int) executes...\n";
    }
    ~Derived() { cout << "~Derived() executes...\n"; }
};

int main() {
    Derived d(5);
    return 0; }
}
```

D:\Edu-CPP\chap12\ex12-05C.exe

```
Base(int) executes...
Base::x = 5
Derived(int) executes...
~Derived() executes...
~Base() executes...
```

多继承和组合类的例子



◆ 若基类只有有参构造函数且参数没有默认值，且需要给基类初始化，则须利用带初始化列表的派生类构造函数，向基类构造函数传参

```
class Base1 {           // 基类Base1，构造函数有参数
public:
    Base1(int i) { cout << "Base1(int) " << i << endl; }
    ~Base1() { cout << "~Base1()" << endl; }
};
```

```
class Base2 {           // 基类Base2，构造函数有参数
public:
    Base2(int i) { cout << "Base2(int) " << i << endl; }
    ~Base2() { cout << "~Base2()" << endl; }
};
```

```
class Base3 {           // 基类Base3，构造函数无参数
public:
    Base3() { cout << "Base3() *" << endl; }
    ~Base3() { cout << "~Base3()" << endl; }
};
```

```
class Derived: public Base2, public Base1, public Base3 {
public:
    Derived(int a, int b, int c, int d):Base1(a), b2(b), b1(c), Base2(d)
    { cout << "Derived(int, int, int, int) ..." << endl; }
    ~Derived() { cout << "~Derived()" << endl; }
```

```
private:
    Base1 b1;
    Base2 b2;
    Base3 b3;
};
```

```
int main() {
    Derived obj(1, 2, 3, 4);
    return 0;
}
```

注意成员对象名的个数与顺序

与初始化列表中的顺序无关

注意基类名的个数与顺序

D:\Edu-CPP\chap12\ex12-06.exe

```
Base2(int) 4
Base1(int) 1
Base3() *
Base1(int) 3
Base2(int) 2
Base3() *
Derived(int, int, int, int) ...
~Derived()
~Base3()
~Base2()
~Base1()
~Base3()
~Base1()
~Base2()
```

拷贝构造函数



- ◆ 若派生类没编写拷贝构造函数，则编译器生成默认拷贝构造函数，该函数先调用基类的拷贝构造函数，再为派生类新增成员对象执行拷贝
- ◆ 若编写派生类复制构造函数，则需要为基类拷贝构造函数传递参数
 - 例如: `Derived:: Derived(const Derived &d1): Base(d1) {...}`

```
class Base{
int i;
public:
    Base(int k = 0):i(k) { }
    show() {cout << "Base::i = " << i << "\n";}
};

class Derived : public Base{
int j;
public:
    Derived(int x = 2, int y = 2) : Base(x), j(y) { }
    Derived(const Derived &obj) : Base(obj), j(obj.j) { }
};
```

若省略Base(obj)
会怎么样?

```
int main()
{
    Derived d1(3, 3);
    Derived d2(d1);
    d2.show();
    return 0;
}
```

不省略Base(obj)时:

```
D:\Edu-CPP\chap12\ex12-07.exe
Base::i = 3
```

省略Base(obj)时:

```
D:\Edu-CPP\chap12\ex12-07.exe
Base::i = 0
```

若省略，创建d2时，d2中派生类成员j是从d1中拷贝过来的，而基类成员i却是默认值，这违背了拷贝应该拷贝整个对象的初衷

本章主要内容



12.1 继承与派生

12.2 继承下的访问控制权

12.3 类的兼容性

12.4 派生类的构造函数与析构函数

12.5 派生类同名成员



12.6 虚基类——共享继承

12.7 实例：小型公司人员信息管理

12.8 作业与实践

函数重写



- ◆ 函数重写（**Function Overriding**）是面向对象编程中的一个概念，在子类中定义一个与父类中同名的函数，以覆盖（重写）父类中的函数实现
- ◆ 函数重写常用于实现多态性（**Polymorphism**），即同一个函数名可以在不同的类中具有不同的行为。通过函数重写，子类可以**改变继承自父类的函数的行为**，以满足子类特定的需求

派生类中同名成员问题



◆当派生类中有同名成员时：

- 若派生类新增成员与基类成员同名，则通过派生类对象使用的是派生类中的同名成员，**隐藏**了基类同名成员（**同名隐藏**）
- 若需通过派生类对象访问基类中被隐藏的同名成员，应使用基类名和作用域操作符（::）来限定（例如：`d.Base1::var`）
- 若来源于**不同基类**的成员同名，通过派生类对象访问同名成员会产生二义性，使用基类名和作用域操作符（::）来限定



多个父类具有同名成员的二义性

- ◆ 消除二义性方案一：使用作用域操作符::来指明基类
- ◆ 消除二义性方案二：**同名隐藏**，即在派生类中声明同名成员

```
class Base1 {  
public:  
    int var;  
    void fun() { cout << "Member of Base1" << endl; }  
};
```

```
class Base2 {  
public:  
    int var;  
    void fun() { cout << "Member of Base2" << endl; }  
};
```

```
class Derived : public Base1, public Base2 {  
public: 方案二  
    int var;  
    void fun() { cout << "Member of Derived" << endl; }  
};
```

```
int main()  
{  
    Derived d;  
    Derived *p = &d;
```

```
    d.var = 1; ✗  
    d.fun(); ✗
```

```
    d.Base1::var = 2; 方案一  
    d.Base1::fun();
```

```
    p->Base2::var = 3;  
    p->Base2::fun();
```

```
    return 0;  
}
```

D:\Edu-CPP\chap12\ex12-08.exe

```
Member of Derived  
Member of Base1  
Member of Base2
```



菱形继承的二义性

◆在多重继承时，多个基类又有共同基类，则在访问此共同基类中的成员时，将产生二义性

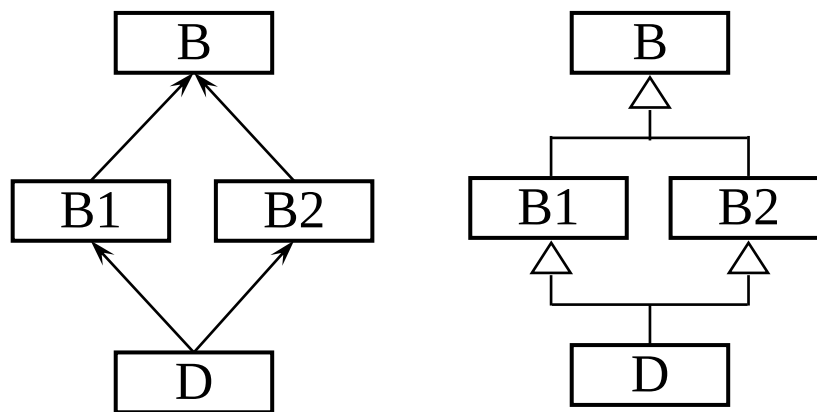
◆一个派生类多次继承同一个基类

■复制继承

➤被多次重复继承的基类有**多个**实体副本，因此，复制继承会产生冗余和二义性

■共享继承

➤被多次重复继承的基类只有**一个**实体副本



菱形继承



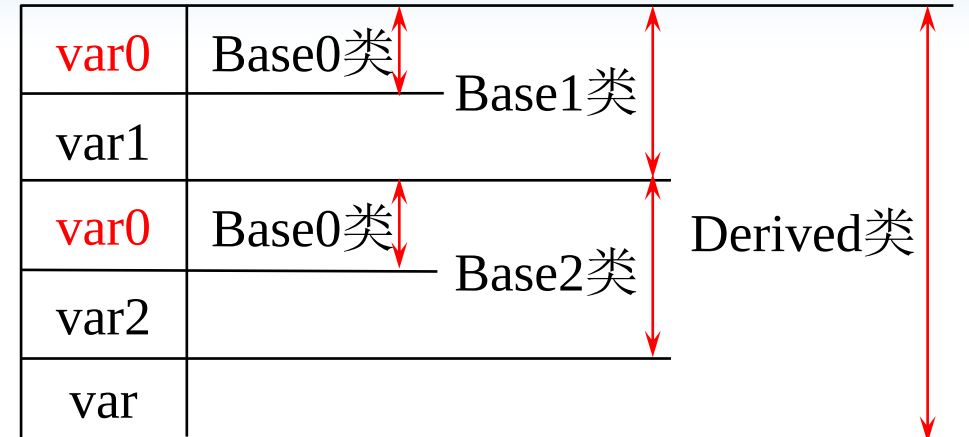
◆ 复制继承会产生冗余和二义性

```
class Base0 {
public:
    int var0;
    void fun0( ) {cout << "Member of Base0" << endl;}
};

class Base1 : public Base0 {
public: int var1;
};

class Base2 : public Base0 {
public: int var2;
};

class Derived : public Base1, public Base2{
public: int var;
    void fun( ) { cout << "Member of Derived" << endl; }
};
```



有二义性:
Derived d;
d.var0;
d.Base0::var0;

无二义性:
d.Base1::var0;
d.Base2::var0;

```
int main() {
    Derived d;
    d.Base1::var0 = 2;
    d.Base1::fun0( );
    d.Base2::var0 = 3;
    d.Base2::fun0( );
    return 0;
}
```

D:\Edu-CPP\chap12\ex12-09.exe

```
Member of Base0
Member of Base0
```

本章主要内容



12.1 继承与派生

12.2 继承下的访问控制权

12.3 类的兼容性

12.4 派生类的构造函数与析构函数

12.5 派生类同名成员

12.6 虚基类——共享继承

12.7 实例：小型公司人员信息管理

12.8 作业与实践



共享继承与虚基类



◆ 共享继承用虚基类机制保证任何派生类中只有一个基类的副本

```
class Base0 {  
public:  
    int var0;  
    void fun0() {cout << "Member of Base0" << endl;}  
};
```

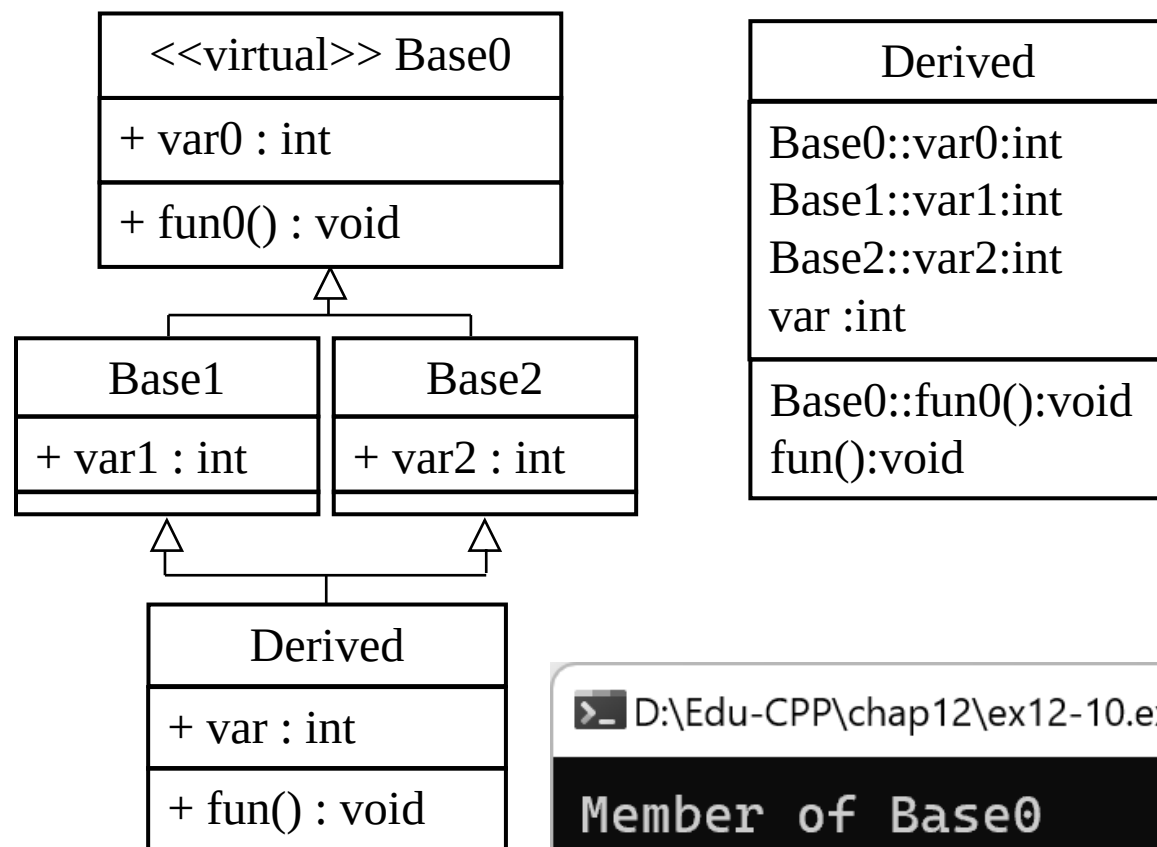
```
class Base1 : virtual public Base0 {  
public: int var1;  
};
```

```
class Base2 : virtual public Base0 {  
public: int var2;  
};
```

```
class Derived : public Base1, public Base2 {  
public: int var;  
    void fun() { cout << "Member of Derived" << endl; }  
};
```

```
int main( )  
{  
    Derived d;  
    d.var0 = 2;  
    d.fun0();  
    return 0;  
}
```

虚基类与普通基类之间的唯一区别只有在派生类重复继承了某一基类时才表现出来，虚基类用于实现共享继承



虚基类



◆形式：**class 派生类名 : virtual 访问控制 基类名**

◆目的：消除二义性

◆说明：

- 虚基类构造函数的调用总是先于普通基类的构造函数

- 虚基类的唯一副本**只被初始化一次**

- 为了保证虚基类对象只被初始化一次，**只在创建对象**的派生类的构造函数中调用虚基类的构造函数，而忽略该派生类的各基类构造函数中对虚基类构造函数的调用

◆C++中构造函数的调用次序（**与初始化列表中的次序无关**）

- (1) 最先调用**虚基类**的构造函数

- (2)其次调用**普通基类**的构造函数，多个基类则按**派生类声明时列出的次序**、从左到右调用

- (3)再次调用**对象成员**的构造函数，按**类声明中对象成员出现的次序**调用

- (4) 最后执行**派生类**的构造函数

共享继承



```
class Base{
public:
    int x;
    Base(){ cout << "Constructing Base " << endl; }
};

class Derived1 : public virtual Base{
public:
    Derived1() { cout << "Constructing Derived1 " << endl; }
};

class Derived2 : virtual public Base{
public:
    Derived2() { cout << "Constructing Derived2 " << endl; }
};

class Derived : public Derived1, public Derived2{
public:
    Derived() { cout << "Constructing Derived " << endl; }
};
```

```
int main() {
    Derived z;
    z.Derived1::x = 1;
    z.Derived2::x = 2;
    cout << z.Derived1::x << endl;
    cout << z.Derived2::x << endl;
}
```

虚基类时

D:\Edu-CPP\chap12\ex12-11.exe

```
Constructing Base
Constructing Derived1
Constructing Derived2
Constructing Derived
2
2
```

非虚基类时

D:\Edu-CPP\chap12\ex12-11B.exe

```
Constructing Base
Constructing Derived1
Constructing Base
Constructing Derived2
Constructing Derived
1
2
```

为什么?

本章主要内容



12.1 继承与派生

12.2 继承下的访问控制权

12.3 类的兼容性

12.4 派生类的构造函数与析构函数

12.5 派生类同名成员

12.6 虚基类——共享继承

12.7 实例：小型公司人员信息管理

12.8 作业与实践



小型公司人员信息管理



◆某小型公司有四类人员：销售人员、技术人员、销售经理和经理

◆要求：

■存储姓名、编号、级别、当月薪水，计算月薪总额、显示个人全部信息

➢注：人员编号基数为1000，每增加一个人员，编号顺序加1

■有提升级别的功能。为简单起见，所有人员的初始级别均为1级，然后进行升级，经理为4级，技术人员和销售经理为3级，销售人员1级

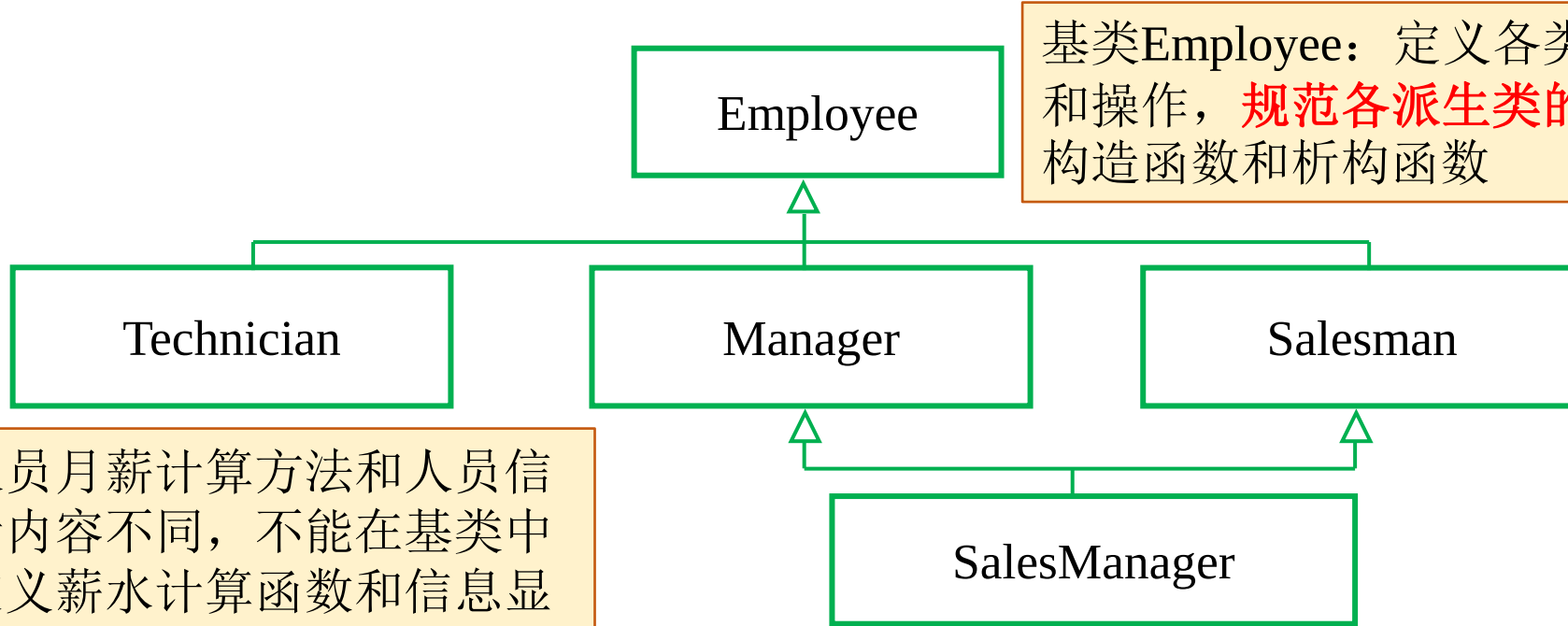
■月薪计算办法是：

➢经理——固定月薪8000元；

➢技术人员——月薪计算100元/小时；

➢销售人员——当月销售额的4%提成；

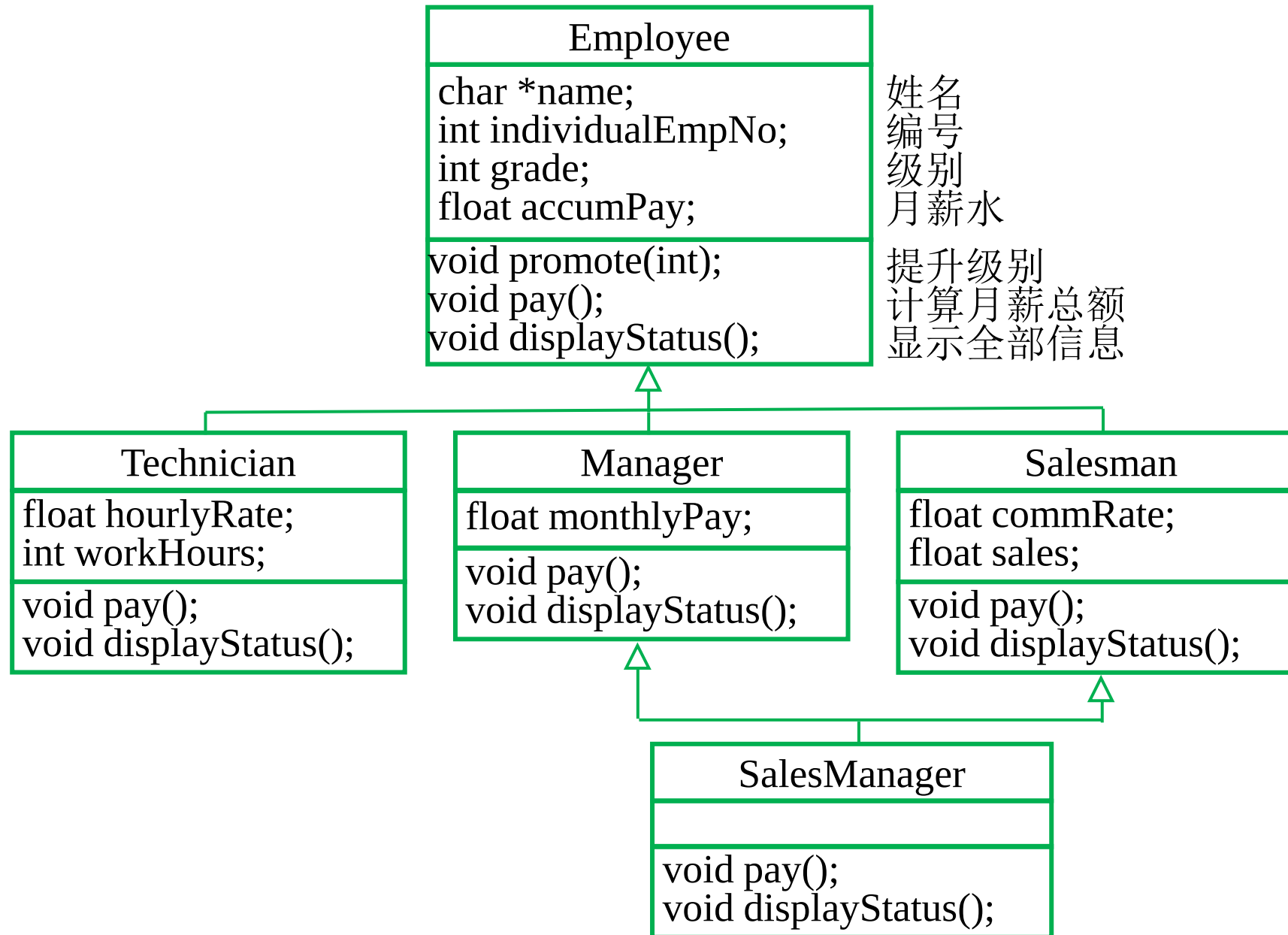
➢销售经理——固定月薪为5000元+销售提成（所管辖部门当月销售总额的5‰）



基类Employee: 定义各类人员共同的信息和操作, **规范各派生类的基本行为**; 定义构造函数和析构函数

各类人员月薪计算方法和人员信息显示内容不同, 不能在基类中统一定义薪水计算函数和信息显示函数, 但可以在基类里规范函数接口, 在**派生类中重新定义**

派生类SalesManager: 两个基类Manger和Saleman, 它们又有公共基类Employee, 为避免二义性, Employee类被设计为**虚基类**



```

#ifndef EMPLOYEE_H
#define EMPLOYEE_H

class Employee
{
protected:
    char *name;
    int individualEmpNo;
    int grade;
    float accumPay;
    static int employeeNo;
public:
    Employee();
    ~Employee();
    void promote(int);
    void pay();
    void displayStatus();
};

#endif

```

Employee.h

```

#include<iostream>
#include<string.h>
#include "Employee.h"
using namespace std;

// 职工编号基数为1000
int Employee::employeeNo = 1000;

Employee::Employee()
{
    char str[30];
    cout << "please input employee's name: ";
    cin >> str;
    name = new char[strlen(str)+1];
    strcpy(name, str);
    individualEmpNo = ++employeeNo;
    grade = 1;
    accumPay = 0;
}

```

Employee.cpp

```

Employee::~Employee()
{
    if(NULL != name)
        delete[] name;
}

void Employee::promote(int incre)
{
    grade += incre;
}

void Employee::pay() { }

void Employee::displayStatus() { }

```

```

#ifndef TECHNICIAN_H
#define TECHNICIAN_H

#include "Employee.h"

// 技术人员
class Technician: public Employee
{
private:
    float hourlyRate;
    int workHours;
public:
    Technician();
    void pay();
    void displayStatus();
};

#endif

```

Technician.h

```

#include<iostream>
#include "Technician.h"
using namespace std;

Technician::Technician()
{
    hourlyRate = 100;
    promote(2);
}

void Technician::pay()
{
    cout << "input " << name << " the workHours of this month: ";
    cin >> workHours;
    accumPay = hourlyRate * workHours;
    cout << "Technician: " << name << ", No: " << individualEmpNo
        << ", month salary: " << accumPay << endl;
}

void Technician::displayStatus() {
    cout << "Technician: " << name << ", No: " << individualEmpNo
        << ", grade: " << grade << ", this month salary: " << accumPay << endl;
}

```

Technician.cpp

```

#ifndef MANAGER_H
#define MANAGER_H

#include "Employee.h"

// 经理
class Manager: virtual public Employee
{
protected:
    float monthlyPay;
public:
    Manager();
    void pay();
    void displayStatus();
};

#endif

```

Manager.h

```

#include<iostream>
#include "Manager.h"
using namespace std;

Manager::Manager()
{
    monthlyPay = 8000;
    promote(3);
}

void Manager::pay()
{
    accumPay = monthlyPay;
    cout << "Manager: " << name << ", No: " << individualEmpNo
        << ", month salary: " << accumPay << endl;
}

void Manager::displayStatus()
{
    cout << "Manager: " << name << ", No: " << individualEmpNo
        << ", grade: " << grade << ", this month salary: " << accumPay << endl;
}

```

Manager.cpp

```

#ifndef SALESMAN_H
#define SALESMAN_H

#include "Employee.h"

// 销售人员
class Salesman: virtual public Employee
{
protected:
    float commRate;
    float sales;
public:
    Salesman();
    void pay();
    void displayStatus();
};

#endif

```

Salesman.h

```

#include<iostream>
#include "Salesman.h"
using namespace std;

Salesman::Salesman()
{
    commRate = 0.05;
}

void Salesman::pay()
{
    cout << "input " << name << " the sales of this month: ";
    cin >> sales;
    accumPay = sales * commRate; // 月薪=销售提成
    cout << "Salesman: " << name << ", No: " << individualEmpNo
        << ", month salary: " << accumPay << endl;
}

void Salesman::displayStatus() {
    cout << "Salesman: " << name << ", No: " << individualEmpNo
        << ", grade: " << grade << ", this month salary: " << accumPay << endl;
}

```

Salesman.cpp

```

#ifndef SALESMANAGER_H
#define SALESMANAGER_H

#include "Manager.h"
#include "Salesman.h"

// 销售经理
class SalesManager: public Manager,
public Salesman
{
public:
    SalesManager();
    void pay();
    void displayStatus();
};

#endif

```

SalesManager.h

```

#include<iostream>
#include "SalesManager.h"
using namespace std;

SalesManager::SalesManager() {
    monthlyPay = 5000;
    commRate = 0.005;
    promote(2);
}

void SalesManager::pay() {
    cout << "input " << Employee::name
        << " the total sales of the department of this month: ";
    cin >> sales;
    accumPay = monthlyPay + commRate * sales;
    cout << "salesman: " << name << ", No: " << individualEmpNo
        << ", month salary: " << accumPay << endl;
}

void SalesManager::displayStatus() {
    cout << "SalesManager: " << name << ", No: " << individualEmpNo
        << ", grade: " << grade << ", this month salary: " << accumPay << endl;
}

```

SalesManager.cpp


```

#include "Manager.h"
#include "Technician.h"
#include "Salesman.h"
#include "SalesManager.h"

int main() {
    Manager m1;
    Technician t1;
    Salesman s1;
    SalesManager sm1;
    m1.promote(3);      // 经理m1升3级
    m1.pay();           // 计算经理m1的月薪
    m1.displayStatus();
    t1.promote(2);      // 技术人员t1升2级
    t1.pay();           // 计算技术人员t1的月薪
    t1.displayStatus();
    s1.pay();           // 计算销售人员s1的月薪
    s1.displayStatus();
    sm1.promote(2);     // 销售经理升2级
    sm1.pay();          // 计算销售经理sm1月薪
    sm1.displayStatus();
    return 0;
}

```

APP.cpp

D:\Edu-CPP\chap12\ex12-12\ex12-12.exe

```

please input employee's name: zhang
please input employee's name: wang
please input employee's name: liu
please input employee's name: xu
Manager: zhang, No: 1001, month salary: 8000
Manager: zhang, No: 1001, grade: 7, this month salary: 8000
input wang the workHours of this month: 240
Technician: wang, No: 1002, month salary: 24000
Technician: wang, No: 1002, grade: 5, this month salary: 24000
input liu the sales of this month: 100000
Salesman: liu, No: 1003, month salary: 5000
Salesman: liu, No: 1003, grade: 1, this month salary: 5000
input xu the total sales of the department of this month: 300000
salesman: xu, No: 1004, month salary: 6500
SalesManager: xu, No: 1004, grade: 8, this month salary: 6500

```

本章主要内容



12.1 继承与派生

12.2 继承下的访问控制权

12.3 类的兼容性

12.4 派生类的构造函数与析构函数

12.5 派生类同名成员

12.6 虚基类——共享继承

12.7 实例：小型公司人员信息管理

12.8 作业与实践



第12章作业



(1)定义一个基类Base类，有成员函数fn1()和fn2()，从它派生出Derived类，重新定义了成员函数fn1()和fn2()；在主函数中声明Derived类对象，分别用Derived的类对象、Base类指针和Derived类指针调用fn1()和fn2()，观察运行结果

(2) ①设计人员类Person

数据成员：姓名、性别、年龄

成员函数：构造函数和void printInfo()（输出数据成员）；

要求：姓名定义为char*，创建类对象时要利用构造函数为new一个空间存放姓名，并用指针指向。类对象生命期结束时，要利用析构函数释放空间

②从Person类派生出学生类Student

新增数据成员：专业、年级、学号

新增成员函数：构造函数和void printInfo()（输出数据成员）

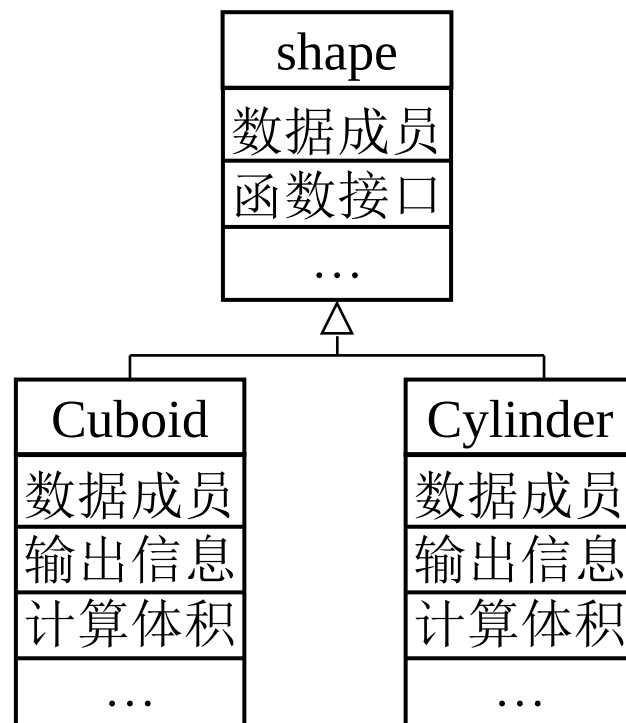
要求：利用一个静态数据成员为类对象自动生存学号

实现两个类，并编写主函数测试父类和子类类之间的同名函数隐藏特性



(3)如图所示类结构，在充分体现多态特性的前提下，设计长方体类Cuboid与圆柱体类Cylinder，实现如下两个功能：

- ①输出信息：在Cuboid类中实现对立方体信息（长、宽、高）的输出；在Cylinder类中实现对圆柱体信息（半径、高）的输出；
- ②计算体积：分别在两个类中实现对长方体和圆柱体体积的计算。





要求:

- ①对长方体类Cuboid和圆柱体类Cylinder共性进行抽象，构成它们共同的基类Shape，即把长方体类Cuboid与圆柱体类Cylinder共性的数据（高）定义为Shape类的数据成员；共性的功能函数（输出信息、计算体积）在Shape类里统一接口，构成shape类的成员函数；
- ②在两个派生类中分别给出功能函数（输出信息、计算体积）的具体实现；
- ③在主函数中通过**动态绑定**的方式，利用基类指针实现对派生类中成员函数（输出信息、计算体积）的调用；
- ④对部分命名进行如下规定：输出信息函数名为dispInfo，计算体积函数名为compVol；长、宽、高、半径分别命名为len、wid、hig、rad；
- ⑤为了配合功能实现，自行增加其他类成员，如构造函数、对数据成员进行赋值的函数等